SET

- If we want to represent a group of unique values as a single entity, then we should go for set.
- Duplicates are not allowed.
- Insertion order is not preserved. But we can sort the elements.
- Indexing and slicing not allowed for the set. ϖ Heterogeneous elements are allowed.
- Set objects are mutable i.e once we create set object we can perform any changes in that object based on our requirement.
- We can represent set elements within curly braces and with comma separation
- We can apply mathematical operations like union, intersection, difference etc. on set objects.

Creation of Set Objects:

(1) s= {10,20,30,40}
print(s)
print(type(s))

```
s={10,20,30,40}
print(s)
print(type(s))
```

```
{40, 10, 20, 30}
<class 'set'>

=== Code Execution Successful ===
```

(2) l = [10,20,30,40,10,20,10]
s=set(l)
print(s)

```
l = [10,20,30,40,10,20,10]
s=set(l)
print(s)
```

```
{40, 10, 20, 30}

=== Code Execution Successful ===
```

(3) s=set(range(5))
print(s)

```
s=set(range(5))
print(s)
```

```
{0, 1, 2, 3, 4}

=== Code Execution Successful ===
```

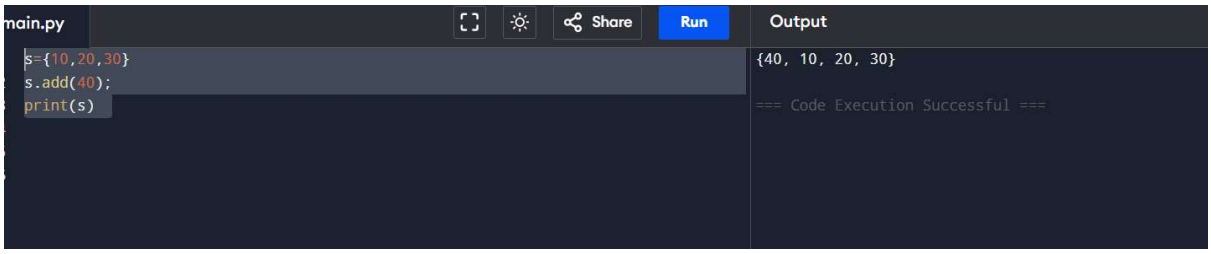
Note:
While creating empty set we have to take special care.
Compulsory we should use set() function.
s = {} → It is treated as dictionary but not empty set.

```
s={}
print(s)
print(type(s))
```

Important Functions of Set:

1. add(x): Adds item x to the set.

```
s={10,20,30}
s.add(40);
print(s)
```



2. **update(x,y,z):**

- **To add multiple items to the set.**
- **Arguments are not individual elements and these are Iterable objects like List, Range etc.**
- **All elements present in the given Iterable objects will be added to the set.**

```
s={10,20,30}
l=[40,50,60,10]
s.update(l,range(5))
print(s)
```

```
s={10,20,30}
l=[40,50,60,10]
s.update(l,range(5))
print(s)
```

```
{0, 1, 2, 3, 4, 40, 10, 50, 20, 60, 30}

=== Code Execution Successful ===
```

3. copy():
   - Returns copy of the set.
   - It is cloned object.

```
s = {10,20,30}
s1 = s.copy()
print(s1)
```

```
s = {10,20,30}
s1 = s.copy()
print(s1)
```

```
{10, 20, 30}

=== Code Execution Successful ===
```

4. pop():
   It removes and returns some random element from the set.

```
s={40,10,30,20}
print(s)
print(s.pop())
print(s)
```

```
main.py
s={40,10,30,20}
print(s)
print(s.pop())
print(s)
```

```
Output
{40, 10, 20, 30}
40
{10, 20, 30}

=== Code Execution Successful ===
```

5. remove(x):
   - It removes specified element from the set.
   - If the specified element not present in the Set then we will get KeyError.

```
s = {40, 10, 30, 20}
s.remove(30)
print(s)
#s.remove(50)
```

```
{40, 10, 20}

=== Code Execution Successful ===
```

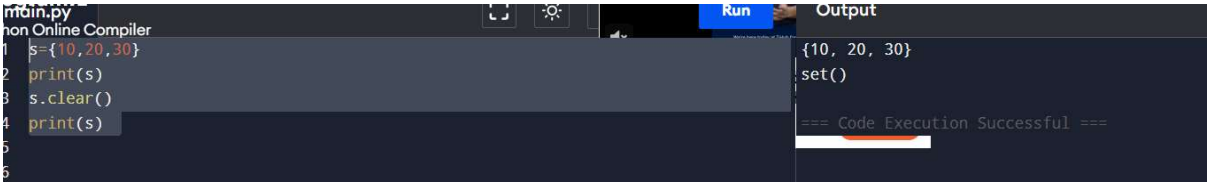
6. discard(x):
   1) It removes the specified element from the set.
   2) If the specified element not present in the set then we won't get any error.

```
s = {40, 10, 30, 20}
s.discard(30)
print(s)
```

7. clear():

To remove all elements from the Set.

```
s={10,20,30}
print(s)
s.clear()
print(s)
```

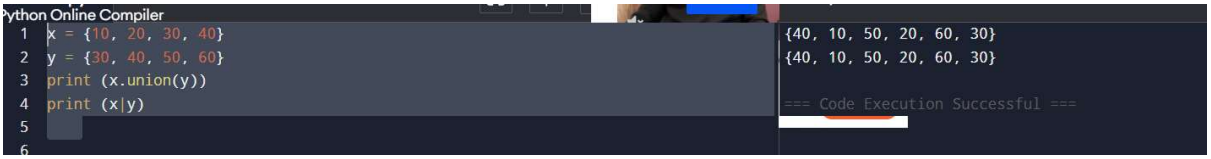


Mathematical Operations on the Set:

(1) union():

- x.union(y) → We can use this function to return all elements present in both sets

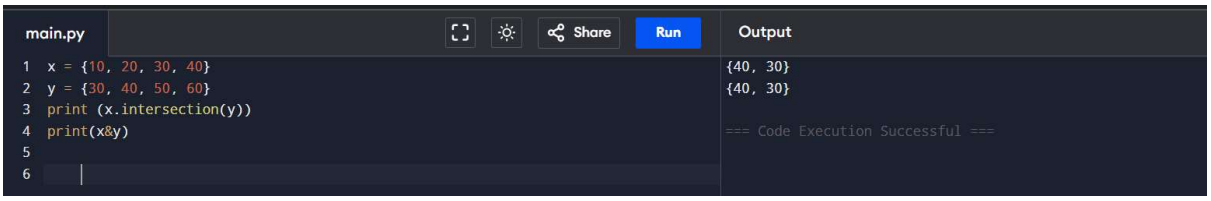
```
x = {10, 20, 30, 40}
y = {30, 40, 50, 60}
print (x.union(y))
print (x|y)
```



(2) intersection():

- x.intersection(y) OR x&y.
- Returns common elements present in both x and y.

```
x = {10, 20, 30, 40}
y = {30, 40, 50, 60}
print (x.intersection(y))
print(x&y)
```

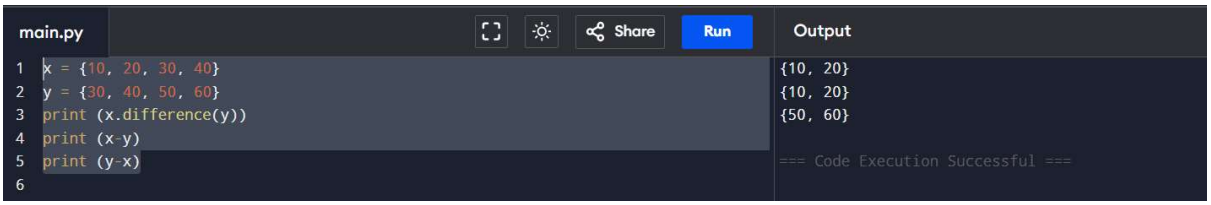


(3) difference():

x.difference(y) OR x-y.

- Returns the elements present in x but not in y.

```
x = {10, 20, 30, 40}
y = {30, 40, 50, 60}
print (x.difference(y))
print (x-y)
print (y-x)
```



(4) symmetric_difference():

x.symmetric_difference(y) OR x^y.

- Returns elements present in either x OR y but not in both.

```
x = {10, 20, 30, 40}
y = {30, 40, 50, 60}
print (x.symmetric_difference(y))
print(x^y)
```

Membership Operators: (in, not in)

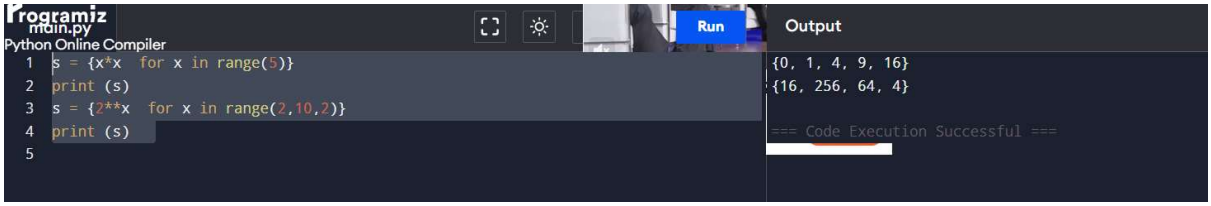
```
s=set("python")
print(s)
print('d' in s)
print('z' in s)
```

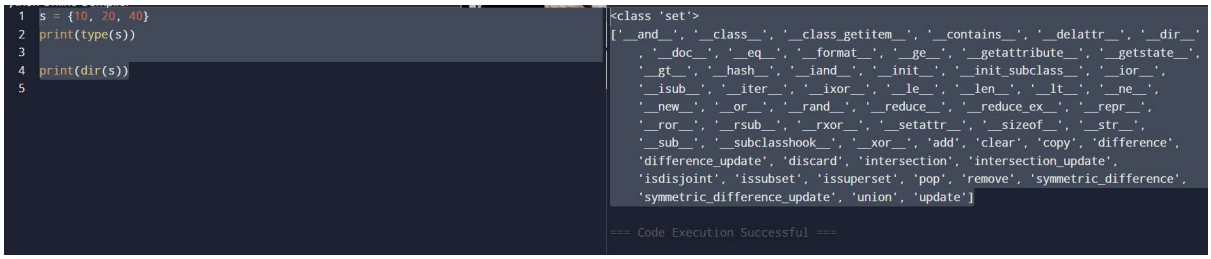
Set Comprehension:

Set comprehension is possible.

```
s = {x*x  for x in range(5)}
print (s)
```

```
s = {2**x  for x in range(2,10,2)}
print (s)
```



```
s = {10, 20, 40}
print(type(s))
print(dir(s))
```

```
<class 'set'>
['__and__', '__class__', '__class_getitem__', '__contains__',
'__delattr__', '__dir__', '__doc__', '__eq__', '__format__',
'__ge__', '__getattribute__', '__getstate__', '__gt__',
'__hash__', '__iand__', '__init__', '__init_subclass__',
'__ior__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__',
'__lt__', '__ne__', '__new__', '__or__', '__rand__',
'__reduce__', '__reduce_ex__', '__repr__', '__ror__',
'__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__',
'__sub__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy',
'difference', 'difference_update', 'discard', 'intersection',
'intersection_update', 'isdisjoint', 'issubset', 'issuperset',
'pop', 'remove', 'symmetric_difference',
'symmetric_difference_update', 'union', 'update']
```